

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	CivicProof	팀 인 원 (팀장 포함)	1명
참가부문	학생	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	CivicProof Ledger 개인정보 비공개 중복수혜 검증 원장
프로젝트 등록 URL	https://github.com/leeminsuk/civicproof-ledger
시연영상	(제출 전 유튜브 업로드 후 URL 기재) ※ 라이브 데모: https://leeminsuk.github.io/civicproof-ledger/
프로젝트 소개	공모전·장학금·복지 바우처 등 공공지원 사업의 중복수혜를 기관 간 원본 개인정보 공유 없이 검증하는 오픈소스 블록체인 원장으로, 부정 중복수혜와 개인정보 유출 위험이라는 국민생활 속 사회문제를 기술로 해결한다. 프로그램별로 격리된 널리파이어(nullifier)와 Ed25519 서명 검증가능 자격증명(VC)으로 “중복은 막고 개인정보는 숨기며”, 공개 감사 이벤트 로그를 누구나 리플레이 재검산(Replay-Verify)하고 시민 무결성 지수(Civic Integrity Index) 0~100점으로 사업 건전성을 정량화한다.
프로젝트 세부 내용	
개발배경 및 목적	1) 문제 정의: 중복수혜 검증과 개인정보 보호의 딜레마 중앙정부·지자체·대학·재단이 운영하는 공모전, 장학금, 연구지원, 복지 바우처는 한정된 예산을 공정하게 배분하기 위해 “동일인·동일 프로젝트의 중복 수혜”를 걸러내야 한다. 그러나 현행 검증은 기관 간 주민등록번호·연락처·계좌 등 원본 개인정보 대조에 의존한다. 이는 개인정보보호법의 최소수집·목적제한 원칙과 충돌하고, 기관 간 데이터 공유 협약·권한관리·보관기간 관리 비용을 발생시키며, 유출 시 파급이 크다. 실제로 본 대회 운영규정 제14조 역시 출품작의 중복 수혜 검증을 요구한다. 즉 검증의 필요성은 보편적이지만, 개인정보를 지키면서 검증하는 도구는 부재하다. 2) 접근: 원본은 기관 로컬에, 공개 원장에는 증명만 원본 식별자는 각 기관 내부에만 두고, 공개 원장에는 프로그램 단위로 격리된 해시(널리파이어)·커밋먼트·서명 증빙·감사 이벤트만 기록한다. 같은 사람이 같은 프로그램에 다시 신청하면 같은 널리파이어가 나와 중복이 즉시 탐지되지만, 다른 프로그램에서는 다른 값이 생성되어 사업 간 교차 추적이 불가능하다. 3) 목적 ① 개인정보 0필드 공개 원장으로 동일 프로그램 내 중복 신청을 등록 시점에 차단 ② 이벤트 로그만으로 누구나 원장 전체를 재검산할 수 있는 감사 구조 확립 ③ 기관·심사자·시민이 동일한

	숫자를 재현하는 결정적 무결성 지수 제공 ④ Apache-2.0 공개로 벤더 종속 없는 공공 인프라 제안. 4) 목표 대비 달성치(전 항목 저장소에서 재현 가능) 자동 테스트 110개 전부 통과(Vitest 90 + Hardhat 20, 속성 기반 퍼징 포함) · 레드팀 공격 12종 중 12종 차단(Node · 브라우저 이중) · 감사 리플레이 MATCH · 시민 무결성 지수 100/100 EXCELLENT · npm audit 취약점 0건 · 자체 평가 하네스 340점 (통과 기준 110점).
개발환경	· 하드웨어: 일반 노트북/데스크톱(Apple Silicon · x86) — GPU · 전용 블록체인 노드 불필요, 네트워크 차단 상태에서도 전 기능 재현(결정적 폴백) · 언어 · 런타임: TypeScript (ESM), Node.js 22, Solidity 0.8.20, Python 3(평가 하네스) · 블록체인: Hardhat 3 로컬 EVM, @nomicfoundation/hardhat-ethers — 컨트랙트 컴파일 · 배포 · 이벤트 조회 자동화 · 암호 · 증명: @noble/ed25519(VC 서명 · 검증), json-canonicalize(RFC8785 정규화), SHA-256 널리파이어 · 커밋먼트 · 상태루트, Schnorr NIZK 데모, 머클 포함증명 · 프론트엔드: 프레임워크 무의존 정적 HTML/CSS/ESM — GitHub Pages 공개 배포, 서명 모듈은 저장소에 번들(vendored)하여 외부 CDN 의존 0 · 품질 · CI: Vitest 90(fast-check 속성 퍼징 포함) · Mocha/Chai 20, 커버리지 임계 게이트, GitHub Actions 10게이트(테스트→커버리지→빌드→컨트랙트→데모→레드팀→로컬배포→하네스→보안감사→SBOM 검증), Playwright 브라우저 E2E · 개발 도구: git/gh CLI(feature→develop→main 브랜치 전략, PR 단위 검증 머지), SPDX SBOM 자동 재생성(npm run sbom)+CI 신선도 검증, Claude Code(보조 도구 — 활용 범위는 불임2에 명시)
시스템 구성 및 아키텍처	■ 5계층 구조와 역할 ① 발급(Issuer/VC) 계층 — 기관은 원본 개인정보를 로컬에서만 처리하고 programId+subjectId+salt의 SHA-256으로 프로그램별 널리파이어를 생성, RFC8785 정규화 JSON에 Ed25519 서명한 VC를 발급한다. VC에는 issuer DID · subject DID · programId · 널리파이어 · 커밋먼트 · 만료일만 담긴다. ② 원장(Contract) 계층 — Solidity ClaimRegistry가 소유자 관리 발급기관 허가목록(RBAC)으로 등록 권한을 통제한다. 허가된 발급자만 registerClaim을 호출할 수 있고, 0값 해시 · 빈 URI는 InvalidInput으로 차단하며, 기등록 프로그램/널리파이어 쌍은 원본을 덮어쓰지 않고 DuplicateDetected 이벤트와 프로그램별 중복 카운터만 남긴다. 소유권 이전(transferOwnership)으로 멀티싱크 · 거버넌스 이관을 지원한다. ③ 감사(Audit) 계층 — 공개 원장은 개인식별정보 없이 ClaimRegistered · DuplicateDetected · IssuerAuthorizationUpdated 이벤트와 카운터를 노출한다. Replay-Verify 엔진이 이 이벤트만으로 전체 상태를 재구축해 결정적 상태루트(civicproof:state-root:v1, SHA-256 도메인 분리)를 비교하고, 시민 무결성 지수(cii-v1)를 산출한다. ④ 웹(Web) 계층 — 정적 검증기 · 공개 감사 UI. 브라우저에서 VC 붙여넣기 → 정상/중복/만료/위변조 4종 판정, 무결성 지수와 리플레이 결과를 동일 공식으로 재현(브라우저 미리). innerHTML

	대신 textContent 렌더링으로 XSS 원천 차단. ⑤ CI/하네스 계층 — GitHub Actions 10게이트와 Python 평가 하네스(340/110)가 문서의 모든 주장과 코드 상태의 불일치를 자동 탐지한다. ■ 핵심 데이터 흐름 신청자 원본ID → [기관 로컬] salt 결합 SHA-256 → 프로그램별 널리파이어 → VC 서명 발급 → ClaimRegistry 등록(최초: ClaimRegistered / 중복: DuplicateDetected) → 공개 감사 로그 → Replay-Verify 재검산·상태루트 대조 → CII 산출 → 웹 대시보드 공개. ■ 공개/비공개 경계 온체인(공개): programId, 널리파이어 해시, 커밋먼트 해시, 메타데이터 URI, 카운터, 이벤트 — PII 0필드. 오프체인(기관 로컬): 원본 식별자, salt, 발급기관 개인키.
프로젝트 주요기능	프로젝트 상세 내용 A. 프로그램 격리 널리파이어 — 원본 subjectId는 어디에도 저장하지 않고 프로그램별 해시만 사용. 같은 프로그램 재신청은 즉시 탐지, 프로그램 간에는 값이 달라 교차 추적을 구조적으로 차단(레드팀 ATK-07로 검증). B. 서명 VC 발급·검증 — Ed25519 키쌍 생성, RFC8785 정규화 서명, 위변조(TAMPERED)·발급자 불일치(WRONG_ISSUER)·만료(EXPIRED)·형식오류(MALFORMED) 4종 사유 분리 판정. 판정은 전부 결정적 코드로, 같은 입력이면 언제나 같은 결과가 나온다. C. EVM ClaimRegistry — 발급기관 허가목록 RBAC, 소유권 이전, 프로그램별 등록·중복 카운터, 원본 클레임 불변 보장. Hardhat 테스트 20개(배포·권한·회수·불량입력·중복계수·프로그램분리·읽기경로·이벤트 리플레이)로 검증. D. 정적 웹 검증기 + 공개 감사 UI — 설치 없이 브라우저에서 VC 검증과 감사 지표 열람. GitHub Pages로 심사위원이 클론 없이 즉시 확인 가능하며, 재사용 CLI(issue/verify/replay/cii)가 외부 감사 로그 파일도 동일 엔진으로 재검산한다. 차별화 엔진 3종(전부 코드·테스트·시연으로 존재) E. [차별화1] Replay-Verify 엔진 — 공개 이벤트 로그만으로 원장 상태를 재구축해 결정적 상태루트를 비교한다. 카운터 조작, 감사 이벤트 없는 클레임 밀반입, 커밋먼트 교체, 등록 없는 중복 이벤트(순서 위조) 등 7종 위조를 유형별 코드(TOTAL_CLAIMS_MISMATCH 등)로 보고한다. 온체인 이벤트를 그대로 리플레이해 컨트랙트 카운터를 정확히 재현하는 교차 검증 테스트 포함 — “장부를 믿지 말고 다시 계산하라”를 실행 가능하게 만든 것. F. [차별화2] 시민 무결성 지수 CII(cii-v1) — 감사일치 40 + 중복차단 30 + 증빙유효 20 + 개인정보최소화 10의 고정 가중치 결정적 공식. 모델·난수 없이 공개 데이터만으로 누구나 동일 점수를 재현하며, 브라우저 미러가 서버 코드와 동일 값을 산출함을 테스트로 보증. 데모 원장은 100/100 EXCELLENT. G. [차별화3] 레드팀 공격 코퍼스 — “공격자가 ~하면?”을 실행 가능한 시나리오 12종(ATK-01~12)으로 코드화: 증빙 위변조·서명 제거·발급자 키 위조·DID 치환·만료 재사용·중복 재등록·교차 추적·비정상 입력·Schnorr 위조·증명 재사용·머클 위조·감사 로그 위조. 12/12 차단이 아니면 실패 종료하며 CI에 상시 편입 — 방어 약화 회귀가 빌드를 깨뜨린다.

	H. Schnorr NIZK 데모·머클 포함증명 — 비밀값을 노출하지 않는 지식 증명과 감사 재현용 포함증명. 프로덕션은 Semaphore/Noir 회로로 대체함을 문서에 정직하게 명시. 개발 과정(주요 단계) ① 수직 슬라이스 우선: 널리파이어→VC→레지스트리→웹까지 항상 연결된 상태 유지 ② 테스트 게이트: 기능마다 테스트 작성→통과 전 병합 금지(29→86개) ③ 브랜치 규율: feature/* → PR(develop, CI 그린 후 머지) → main 릴리스(v1.0.0 태그), 8개 PR 순차 검증 ④ 보안 보완 이력: 무허가 등록 리스크→발급자 RBAC 도입, 마크업 싱크 배제(createElement·textContent), Pages 서명 모듈 404 결함 발견→저장소 번들로 교정(라이브 데모 정상화). 구동 및 시연 npm ci → npm test(90 passed, 속성 퍼징 포함) → npm run coverage(임계 통과) → npm run build(타입 무결성) → npm run test:contracts(20 passing) → npm run demo(정상 2건·중복 1건 차단·PII 0필드·Replay MATCH·CII 100/100) → npm run redteam(12/12 차단) → npm run deploy:local → npm run evaluate(340/110) → npm audit(0건) → npm run sbom:check. 모든 명령은 외부 API·키 없이 오프라인 재현된다. 웹 시연(4막 인터랙티브 대시보드, https://leeminsuk.github.io/civicproof-ledger/): ① 히어로 — 개인정보가 해시 방벽을 지나 널리파이어로 바뀌는 애니메이션과 ‘온체인 개인정보 0필드’ KPI ② 원장 시뮬레이터 — [시나리오 자동 재생]으로 같은 사업 재신청은 중복 차단, 다른 사업은 널리파이어가 달라 교차추적 불가임을 눈으로 확인 ③ 공격 극장 — [공격 12종 실행]으로 레드팀 ATK-01~12를 브라우저에서 실제 검산해 12/12 차단 ④ 무결성 대시보드 — 시민 무결성 지수 게이지 100/100, [감사 로그 조작] 토크 시 Replay-Verify가 어긋남(EVENT_ORDER_VIOLATION)을 잡아내며 지수가 60 WATCH로 하락 후 복구. 시연영상은 이 4막을 3분 내로 담는다(촬영 콘티: docs/submission/시연영상-촬영가이드.md).
기대효과 및 활용분야	향후 확장성 및 기대효과 · 적용 분야 — 공모전·장학금·연구지원·복지 바우처·지역화폐·탄소포인트 등 “동일 사업 내 1인 1회” 원칙이 필요한 모든 공공·비영리 배분 사업. 발급기관 허가목록이 다기관 연합 운영(중앙+지자체+대학)을 그대로 지원한다. · 개인정보보호 효과 — 기관 간 원본 개인정보 공유 없이 중복 검증이 가능해져 최소수집·목적제한 원칙에 부합. 공개 원장의 PII는 0필드로, 유출될 개인정보 자체가 존재하지 않는다. · 감사·투명성 효과 — “신뢰하지 말고 검증하라”를 공공지원에 적용. 시민 누구나 이벤트 로그를 리플레이해 수리·차단 건수와 상태루트를 재검산하고, 무결성 지수로 사업 건전성을 비교할 수 있다. 이의제기 시 원본이 아닌 서명 증빙으로 검증 흐름을 재현한다. · 오픈소스 생태계 기여 — Apache-2.0으로 공개된 레지스트리·VC 헬퍼·정적 검증기·무결성 지수·레드팀 코퍼스가 각각 독립 모듈이라 개별 재사용이 가능하다. 특히 “공격 코퍼스를 CI 게이트로 두는” 패턴은 타 보안 프로젝트가 바로 가져다 쓸 수 있는 방법론이다. · 시장성 — 공공기관 대상 검증 SaaS, 감사기관용 리플레이 도구,

	기관 온보딩(발급키 관리·정책 설계) 서비스로 확장 가능. 벤더 종속이 없어 도입 장벽이 낮다. 기술 확장 — Semaphore/Noir 영지식 회로로 널리파이어 소유 증명 완전 익명화, 퍼블릭 테스트넷 상시 배포, KMS 발급키 운영, DID resolver 연동, 기관 간 연합 거버넌스로 발전시킨다.
기타	프로젝트의 혁신성 및 차별성 ① 문제 정의의 전환 — “개인정보를 모아야 검증할 수 있다” 는 전제를 뒤집어, 프로그램 격리 널리파이어로 검증과 프라이버시를 동시에 달성했다. 블록체인을 토큰·NFT가 아닌 공공행정의 실제 운영 문제(중복수해)에 적용한 드문 사례다. ② 검증가능성 문화 — 보고서의 모든 수치(테스트 110, 12/12 차단, 340/110, CII 100/100, audit 0건)는 저장소의 명령어 한 줄로 재현된다. 주장과 코드의 불일치는 자체 하네스와 CI가 잡아내고, fast-check 퍼징이 무작위 수백 시나리오로 3대 엔진의 불변식을 상시 재검증한다. ③ 3대 엔진의 유기적 결합 — Replay-Verify(감사 재검산)가 CII의 최대 배점(40점)을 공급하고, 레드팀 코퍼스가 그 방어를 상시 공격해 회귀를 차단한다. 기능 나열이 아니라 서로를 검증하는 구조다. ④ 심사 재현성 설계 — 외부 API·키·GPU 없이 오프라인 전체 재현, GitHub Pages 즉시 시연, SPDX SBOM(205개, 재생성·CI 검증)·라이선스 문서·체크리스트까지 심사·검증 절차 자체를 제품 요건으로 취급했다. ⑤ 대회 규정 정합성 — 직접 작성 코드 전체에 OSI 인증 Apache-2.0 적용, 활용 라이브러리 전수 SBOM 공개(붙임1), AI 모델 미답재로 상용 API 의존·가중치 공개 이슈 원천 부재(붙임2), 공개 저장소 5년 유지 계획, 정부 지원사업 수혜 이력 없음(중복수해 해당 없음). 운영규정 제8~10조·제14조의 요구를 저장소 구조로 충족하며, v1.0.0은 기여 가이드·보안 정책·이슈/PR 템플릿·전 소스 SPDX 헤더·한국어 README까지 갖춰 오픈소스 성숙도를 증빙한다. 한계점 및 향후 발전 로드맵 - 정직한 현재 범위 — 본 출품작은 검증 가능한 최소 운영 모델이다. Ed25519Signature2020Demo는 W3C VC 완전 준수가 아니고, Schnorr NIZK는 데모 수준이며, DID resolver·실기관 KMS 운영은 미구현이다. 이 경계는 README와 아키텍처 문서에 명시되어 있다. - 로드맵 — 2026 H2: Semaphore/Noir ZK 회로 통합(널리파이어 소유 증명 완전 익명화), 퍼블릭 테스트넷 상시 배포 및 컨트랙트 주소 공개, W3C VC Data Model 정합 / 2027 H1: 발급키 KMS·HSM 운영 가이드, 멀티시그 소유권 이관, 파일럿 기관(대학 장학·지자체 공모전) 1곳 실증 / 2027 H2: 다기관 연합 정책 엔진, 이의제기 워크플로, 무결성 지수 공개 대시보드 상용화. 각 단계는 현재의 테스트 게이트·레드팀 코퍼스를 그대로 상속해 회귀 없이 확장한다. 소감 및 후기 1인 팀으로 설계·구현·검증·문서를 감당하며 “주장은 코드로 증명한다” 는 규율을 세웠다. 통과 전 병합 금지 게이트와 레드팀 코퍼스로 AI 보조 코드까지 전량 기계 검증했고, 배포 데모 결함을 직접 고치며 검증이 품질을 만든다는 것을 체감했다. 실제 기관 파일럿까지 발전시키고 싶다.

붙임1

SBOM(소프트웨어 자재명세서)

번호	라이브러리명	버전	라이선스	공식 저장소 URL (GitHub 등)	사용 목적 및 주요 기능
1	@noble/ed25519	2.3.0	MIT	https://github.com/paulmillr/noble-ed25519	Ed25519 서명 · 검증 (VC proof 생성 · 확인), web/vendor에 번들 포함
2	json-canonicalize	2.0.0	MIT	https://github.com/mirkokiefer/canonical-json	RFC8785 호환 JSON 정규화 (서명 대상 직렬화)
3	commander	12.1.0	MIT	https://github.com/tj/commander.js	데모 · 레드팀 CLI 인자 파싱
4	hardhat	3.9.0	MIT	https://github.com/NomicFoundation/hardhat	Solidity 컴파일 · 로컬 EVM · 컨트랙트 테스트 · 배포
5	@nomicfoundation/hardhat-ethers	4.0.13	MIT	https://github.com/NomicFoundation/hardhat	Hardhat-ethers 연동 (컨트랙트 호출 · 이벤트 조회)
6	@nomicfoundation/hardhat-mocha	3.0.21	MIT	https://github.com/NomicFoundation/hardhat	컨트랙트 테스트 러너 통합
7	mocha	11.7.6	MIT	https://github.com/mochajs/mocha	컨트랙트 테스트 프레임워크
8	chai	6.2.2	MIT	https://github.com/chaijs/chai	테스트 어서션 라이브러리
9	vitest	4.1.9	MIT	https://github.com/vitest-dev/vitest	TypeScript 단위 · 통합 테스트 55개 실행
10	typescript	6.0.3	Apache-2.0	https://github.com/microsoft/TypeScript	정적 타입 검사 · 빌드 (strict)
11	tsx	4.22.4	MIT	https://github.com/privatenumber/tsx	TypeScript ESM 실행기 (데모 · 레드팀 구동)
12	@types/node	26.0.0	MIT	https://github.com/DefinitelyTyped/DefinitelyTyped	Node.js 타입 정의
13	solc (Hardhat 내장)	0.8.28	GPL-3.0	https://github.com/ethereum/solidity	Solidity 0.8.20 컨트랙트 컴파일 (빌드 도구, 산출물 미배포)

번호	라이브러리명	버전	라이선스	공식 저장소 URL (GitHub 등)	사용 목적 및 주요 기능
14	actions/checkout 등 GitHub Actions	v4/v5	MIT	https://github.com/actions	CI 체크아웃 · Node 설정 · Pages 배포
15	fast-check	4.8.0	MIT	https://github.com/dubzzz/fast-check	property 기반 퍼징 (불변식 무작위 검증) 테스트
16	@vitest/ coverage-v8	4.1.10	MIT	https://github.com/vitest-dev/vitest	커버리지 측정 · 임계 게이트 (dev)

※ 위 표는 직접 의존성 기준이며, 전이 의존성을 포함한 전체 205개 패키지의 SPDX SBOM은 저장소의 sbom.spdx.json으로 공개되며, npm run sbom으로 재생성되고 CI가 잠금파일과의 일치를 상시 검증한다. 전 패키지 npm audit 취약점 0건.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ■ 표시)

- ☐ 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
 - ☐ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정한 경우)
 - ☐ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
- ※ 개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI(GPT, Claude 등)를 단순 활용한 경우는 체크하지 않음(4번 항목에 기재)
- ※ 해당 없음 — 본 출품작은 AI 모델을 탑재·적용하지 않음. 중복 판정·서명 검증·무결성 지수 산출은 전부 결정적 코드로 수행되며, 어떤 추론 모델도 포함하지 않는다.

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음(AI 모델 미탑재)	기반 모델 라이선스	해당 없음
--------------	------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	해당 없음(학습 데이터셋 미사용)
데이터 정제/ 가공 방법 요약	해당 없음
새로 생성된 가중치 공개 저장소 URL	해당 없음
가중치 파일 정보 및 배포방식	해당 없음

4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)			
직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0 (OSI 인증)	학습/추론 소스코드 공개 저장소 URL	https://github.com/leeminsuk/civicproof-ledger (공개 유지)
상용 AI 보조도구 활용 여부 및 범위	코드 작성 · 리팩터링 · 디버깅 보조용으로 Anthropic Claude Code를 활용함. AI 보조로 작성된 코드는 전량 자동 테스트 110개 (속성 기반 퍼징 포함), 레드팀 공격 코퍼스 12종, CI 10단계 게이트로 검증했으며, 핵심 동작 원리(널리파이어 격리, RFC8785 서명, 리플레이 상태루트, cii-v1 공식)는 docs/architecture.md에 직접 기술하여 이해도를 증빙한다. 출품작 자체에 탑재된 AI 모델은 없다.		